**German University in Cairo**
**Media Engineering and Technology**
**Prof. Dr. Slim Abdennadher**
**Dr. Nourhan Ehab**
**Dr. Ahmed Abdelfattah**

**CSEN 401 - Computer Programming Lab**, *Spring 2026*
DooR DasH: Scare vs Laugh Touchdown
**Milestone 1**
*Deadline: 12.3.2026 @ 11:59 PM*

This milestone is an *exercise* on the concepts of **Object Oriented Programming (OOP)**. The following sections describe the requirements of the milestone.

By the **end of this milestone**, you should have:

- A structured package hierarchy for your code.
- An initial implementation for all the needed data structures.
- Basic data loading functionality from a csv file.

# 1 Build the Project Hierarchy

## 1.1 Add the packages

Create a new `Java` project and build the following package hierarchy:

1. `game.engine`
2. `game.engine.cards`
3. `game.engine.cells`
4. `game.engine.monsters`
5. `game.engine.dataloader`
6. `game.engine.exceptions`
7. `game.engine.interfaces`
8. `game.tests`

Afterwards, proceed by implementing the following classes. You are allowed to add more classes, attributes and methods. However, you must use the same specific names for the provided ones.

## 1.2 Naming and privacy conventions

Please note that all instance variables in your class attributes must be `private`, all constants must be `public` and all methods should be `public` unless otherwise stated. Your implementation should have the appropriate setters and getters conforming with the access constraints. if a variable is said to be READ then we are allowed to get its value. If the variable is said to be WRITE then we are allowed to change its value. Please note that getters and setters should match the Java naming conventions unless mentioned otherwise. If the instance variable is of type boolean, the getter method name starts by **is**

followed by the **exact** name of the instance variable. Otherwise, the method name starts by the verb (get or set) followed by the **exact** name of the instance variable; the first letter of the instance variable should be capitalized. Please note that the method names are case sensitive.

**Example 1** *You want a getter for an instance variable called* `milkCount` → *Method name =* `getMilkCount()`

**Example 2** *You want a setter for an instance variable called* `milkCount` → *Method name =* `setMilkCount()`

**Example 3** *You want a getter for a boolean variable called* `alive` → *Method name =* `isAlive()`

Throughout the whole milestone, if an attribute will never be changed once initialized, you should declare it as `final`. If a particular member (variable or method) belongs to the class, rather than instances of the class then it's said to be `static`. Combining both is commonly used for defining constants that should be accessible without creating an instance of the class and whose value remains constant throughout the program. Constants are always public and do not require getters nor setters of course.

**Example 4** *You want a constant price value for each instance* → `final int price = 100;`

**Example 5** *You want a counter that will be shared by all instances* → `static int counter = 0;`

**Example 6** *You want a constant shared by all instances* → `public static final int code = 1;`

### 1.3 Comparing Objects

In Java, the Comparable interface is used to impose a natural ordering on the objects of each class that implements it. The Comparable interface defines a single method, `compareTo()`, which is used to compare the current object with another object for order based on some criteria. To use comparable, a class must implement the `Comparable` interface and define the `compareTo()` method. This method takes the object to be compared with the current one and returns an integer that is equal to:

1. A negative integer → current object is less than the specified object.
2. Zero → current object is equal to the specified object.
3. A positive integer → current object is greater than the specified object.

**Example 7** *Suppose you have a* `Cat` *class, and you want to sort cats by their weight*

```
public class Cat implements Comparable<Cat> {
    private String name;
    private int weight;

    public Cat(String name, int weight) {
        this.name = name;
        this.weight = weight;
    }

    public int compareTo(Cat cat) {
        return this.weight - cat.weight;
    }
```

## 2 Build the Interfaces

Interfaces define a set of abstract functionalities that a class can implement, allowing it to perform specific actions, like the comparing mentioned above. Classes should implement the interfaces that allow them to do their specified functionality.

### 2.1 Build the CanisterModifier Interface

**Name** : `CanisterModifier`

**Package** : `game.engine.interfaces`

**Type** : Interface

**Description** : Interface containing the method available to all objects that can modify a monster's energy (canister) within the game.

## 3 Build the Enums

### 3.1 Build the Role Enum

**Name** : `Role`

**Package** : `game.engine`

**Type** : Enum

**Description** : An enum representing the two roles of monsters in the game. Possible values are: `SCARER`, `LAUGHER`.

## 4 Build the Constants Class

A Constant Class is a class that never change once initialized. Its main purpose is to declare and initialize constants. A constant is both: something that never changes after initialized and also available at class level. Constants are always public and are accessible so do not require getters (nor setters of course).

### 4.1 Build the Constants Class

**Name :** `Constants`

**Package :** `game.engine`

**Type :** Final class.

**Description :** A class containing all game constants.

#### 4.1.1 Attributes

Board constants:

- `int BOARD_SIZE = 100`
- `int BOARD_ROWS = 10`
- `int BOARD_COLS = 10`
- `int WINNING_POSITION = 99`
- `int STARTING_POSITION = 0`

Special cells positions constants:

- `int[] MONSTER_CELL_INDICES = {2, 18, 34, 54, 82, 88}`
- `int[] CONVEYOR_CELL_INDICES = {6, 22, 44, 52, 66}` (Start of the belt)
- `int[] SOCK_CELL_INDICES = {32, 42, 74, 84, 98}` (Start of the sock)
- `int[] CARD_CELL_INDICES = {4, 12, 28, 36, 48, 56, 60, 76, 86, 90}`

Energy constants:

- `int WINNING_ENERGY = 1000`
- `int MIN_ENERGY = 0`

Monster constants:

- `int MULTITASKER_BONUS = 200`
- `int SCHEMER_STEAL = 10`

Cell constants:

- `int SLIP_PENALTY = 100`

Power constants:

- `int POWERUP_COST = 500`

# 5 Build the Classes

You need to think about the boundary values of all variables in the setters. For example, you need to make sure that certain variables don't fall below zero or certain values loop around.

## 5.1 Build the Monster Class

**Name :** `Monster`

**Package :** `game.engine.monsters`

**Type :** Abstract class.

**Description :** A class representing the Monsters available in the game. A monster can be compared based on its position on the board.

No objects of type `Monster` can be instantiated.

### 5.1.1 Attributes

All the class attributes are **READ AND WRITE** unless stated otherwise.

1. `String name`: A string representing the monster's name. This attribute is **READ ONLY**.
2. `String description`: A string representing the description of the monster's special power. This attribute is **READ ONLY**.
3. `Role role`: An enum representing the role of the monster (`SCARER` or `LAUGHER`).
4. `Role originalRole`: An enum representing the original role of the monster (`SCARER` or `LAUGHER`). This attribute is **READ ONLY**.
5. `int energy`: An integer representing the amount of energy the monster has collected. (>= 0)
6. `int position`: An integer representing the monster's current position on the board (0-99).
7. `boolean frozen`: A boolean indicating whether the monster is currently frozen.
8. `boolean shielded`: A boolean indicating whether the monster is currently shielded.
9. `int confusionTurns`: An integer representing the number of turns the monster will be confused (becomes the other role)

### 5.1.2 Constructors

1. `Monster(String name, String description, Role originalRole, int energy)`: Constructor that initializes a Monster object with the given parameters as the attributes. The role should start with the originalRole too. The position and confusion turns are initially set to the first index (0). The boolean values are initially set to false.

### 5.1.3 Methods

1. `int compareTo(Monster o)`: override the `Comparable` interface `compareTo` method to order monsters based on their position on the board (who is more closer to the end).

### 5.1.4 Subclasses

The game features four distinct types of monsters, each represented as a subclass within the Monster class hierarchy. These subclasses should be organized in the same package where the Monster class is located. The following list gives the different types of monsters:

- Dasher
- Dynamo
- MultiTasker
- Schemer

## 5.2 Build the Dasher Class

**Name :** `Dasher`

**Package :** `game.engine.monsters`

**Type :** Class.

**Description :** A class representing the Dasher monsters available in the game. This class is a subclass of the Monster class.

### 5.2.1 Attributes

1. `int momentumTurns`: An integer representing the number of turns the Dasher will be having momentum. This attribute is **READ AND WRITE**.

### 5.2.2 Constructors

1. `Dasher(String name, String description, Role role, int energy)`: Constructor that initializes a Dasher object with the given parameters as the attributes by calling the super constructor and sets the momentumTurns to 0.

## 5.3 Build the Dynamo Class

**Name :** `Dynamo`

**Package :** `game.engine.monsters`

**Type :** Class.

**Description :** A class representing the Dynamo monsters available in the game. This class is a subclass of the Monster class.

### 5.3.1 Constructors

1. `Dynamo(String name, String description, Role role, int energy)`: Constructor that initializes a Dynamo object with the given parameters as the attributes by calling the super constructor.

## 5.4 Build the MultiTasker Class

**Name :** `MultiTasker`

**Package :** `game.engine.monsters`

**Type :** Class.

**Description :** A class representing the MultiTasker monsters available in the game. This class is a subclass of the Monster class.

### 5.4.1 Attributes

1. `int normalSpeedTurns`: An integer representing the number of turns the MultiTasker will be moving at normal speed. This attribute is **READ AND WRITE**

### 5.4.2 Constructors

1. `MultiTasker(String name, String description, Role role, int energy)`: Constructor that initializes a MultiTasker object with the given parameters as the attributes by calling the super constructor and sets normalSpeedTurns to 0.

## 5.5 Build the Schemer Class

**Name :** `Schemer`

**Package :** `game.engine.monsters`

**Type :** Class.

**Description :** A class representing the Schemer monsters available in the game. This class is a subclass of the Monster class.

### 5.5.1 Constructors

1. `Schemer(String name, String description, Role role, int energy)`: Constructor that initializes a Schemer object with the given parameters as the attributes by calling the super constructor.

## 5.6 Build the Cell Class

**Name :** `Cell`

**Package :** `game.engine.cells`

**Type :** Class.

**Description :** A class representing the cells on the game board that a monster can land on. This class can be instantiated as normal cells with no special features.

### 5.6.1 Attributes

1. `String name`: A string representing the name of the cell. This attribute is **READ ONLY**.
2. `Monster monster`: A Monster object representing the monster currently landed this cell. This attribute is **READ AND WRITE**.

### 5.6.2 Constructors

1. `Cell(String name)`: Constructor that initializes a Cell object with the given name and sets the monster to `null`.

### 5.6.3 Subclasses

The game features several types of cells. All cell subclasses should be in the `game.engine.cells` package:

- DoorCell
- TransportCell
- MonsterCell
- CardCell

## 5.7 Build the DoorCell Class

**Name :** `DoorCell`

**Package :** `game.engine.cells`

**Type :** Class.

**Description :** Represents doors where monsters collect or lose energy depending on role. Subclass of Cell and can modify canister energy.

### 5.7.1 Attributes

1. `Role role`: The role type (`SCARER` or `LAUGHER`) that this door supports. This attribute is **READ ONLY**.
2. `int energy`: The amount of energy this door provides. This attribute is **READ ONLY**.
3. `boolean activated`: Whether this door has been activated or not. Doors start with deactivated and activates when a monster passes on it. Once its activated, no monster can reuse it. This attribute is **READ AND WRITE**.

### 5.7.2 Constructors

1. `DoorCell(String name, Role role, int energy)`: Initializes with name, role, and energy. `activated` starts as `false`.

## 5.8 Build the MonsterCell Class

**Name :** `MonsterCell`

**Package :** `game.engine.cells`

**Type :** Class.

**Description :** Represents special cells with stationed monsters. Subclass of Cell.

### 5.8.1 Attributes

1. `Monster cellMonster`: The monster stationed at this cell. This attribute is **READ ONLY**.

### 5.8.2 Constructors

1. `MonsterCell(String name, Monster cellMonster)`: Initializes with name by calling the super constructor and sets the stationed monster in the cell monster.

## 5.9 Build the CardCell Class

**Name :** `CardCell`

**Package :** `game.engine.cells`

**Type :** Class.

**Description :** Represents cells where players can draw cards. Subclass of Cell.

### 5.9.1 Constructors

1. `CardCell(String name)`: Constructor that initializes a CardCell object by calling the super constructor.

## 5.10 Build the TransportCell Class

**Name :** `TransportCell`

**Package :** `game.engine.cells`

**Type :** Abstract class.

**Description :** Represent cells that transport monsters. Subclass of Cell and No objects of type `TransportCell` can be instantiated

### 5.10.1 Attributes

1. `int effect`: The transport effect value (positive for forward, negative for backward). This attribute is **READ ONLY**.

### 5.10.2 Constructors

1. `TransportCell(String name, int effect)`: Sets TransportCell with the name by calling the super constructor and sets the effect value.

### 5.10.3 Subclasses

The game features two types of transport cells. All transport cell subclasses should be in the `game.engine.cells` package:

- ConveyorBelt
- ContaminationSock

## 5.11 Build the ConveyorBelt Class

**Name :** `ConveyorBelt`

**Package :** `game.engine.cells`

**Type :** Class.

**Description :** Represents conveyor belts that move monsters. Subclass of TransportCell.

### 5.11.1 Constructors

1. `ConveyorBelt(String name, int effect)`: Constructor that initializes a ConveyorBelt object by calling the super constructor. The effect value is always positive.

## 5.12 Build the ContaminationSock Class

**Name :** `ContaminationSock`

**Package :** `game.engine.cells`

**Type :** Class.

**Description :** Represents contamination socks that penalize monsters both in position and energy. Subclass of TransportCell and can modify energy

### 5.12.1 Constructors

1. `ContaminationSock(String name, int effect)`: Constructor that initializes a ContaminationSock object by calling the super constructor. The effect value is always negative.

## 5.13 Build the Card Class

**Name :** `Card`

**Package :** `game.engine.cards`

**Type :** Abstract class.

**Description :** A class representing the special cards available in the game.

No objects of type `Card` can be instantiated.

### 5.13.1 Attributes

All attributes are **READ ONLY**.

1. `String name`: The card's name.
2. `String description`: The card's description.
3. `int rarity`: The card's rarity level. Specifies the number of cards in the card pile
4. `boolean lucky`: Whether this is a lucky card or not according to the player.

### 5.13.2 Constructors

1. `Card(String name, String description, int rarity, boolean lucky)`: Constructor that initializes a Card object with the given parameters.

### 5.13.3 Subclasses

Card have the following subclasses that should be in `game.engine.cards` package:

- SwapperCard
- EnergyStealCard
- StartOverCard
- ConfusionCard
- ShieldCard

## 5.14 Build the SwapperCard Class

**Name :** `SwapperCard`

**Package :** `game.engine.cards`

**Type :** Class.

**Description :** Represents swapper cards that allow position swapping. Subclass of Card.

### 5.14.1 Constructors

1. `SwapperCard(String name, String description, int rarity)`: Constructor that calls super with the parameters and sets `lucky` to `true`.

## 5.15 Build the EnergyStealCard Class

**Name :** `EnergyStealCard`

**Package :** `game.engine.cards`

**Type :** Class.

**Description :** Represents energy steal cards. Subclass of Card and can modify energy.

### 5.15.1 Attributes

1. `int energy`: The amount of energy to steal. This attribute is **READ ONLY**.

### 5.15.2 Constructors

1. `EnergyStealCard(String name, String description, int rarity, int energy)`: Constructor that calls super with name, description, rarity, and sets `lucky` to `true`. Also initializes the energy attribute.

## 5.16 Build the StartOverCard Class

**Name :** `StartOverCard`

**Package :** `game.engine.cards`

**Type :** Class.

**Description :** Represents start over cards that resets position. Subclass of Card. Can be lucky or unlucky.

### 5.16.1 Constructors

1. `StartOverCard(String name, String description, int rarity, boolean lucky)`: Constructor that calls super with all parameters including the `lucky` value.

## 5.17 Build the ConfusionCard Class

**Name :** `ConfusionCard`

**Package :** `game.engine.cards`

**Type :** Class.

**Description :** Represents confusion cards that confuse monsters of their role. Subclass of Card.

### 5.17.1 Attributes

1. `int duration`: The number of turns the confusion of roles lasts. This attribute is **READ ONLY**.

### 5.17.2 Constructors

1. `ConfusionCard(String name, String description, int rarity, int duration)`: Constructor that calls super with name, description, rarity, and sets `lucky` to `false`. Also initializes the duration attribute.

## 5.18 Build the ShieldCard Class

**Name :** `ShieldCard`

**Package :** `game.engine.cards`

**Type :** Class.

**Description :** Represents shield cards that protect monsters from energy losing. Subclass of Card.

### 5.18.1 Constructors

1. `ShieldCard(String name, String description, int rarity)`: Constructor that calls super with the parameters and sets `lucky` to `true`.

# Game Setup

## 5.19 Build the DataLoader Class

**Name :** `DataLoader`

**Package :** `game.engine.dataloader`

**Type :** Class.

**Description :** A class used to import data from CSV files. The imported data is used to create Cards, Cells, and Monsters. The read file should be placed in the same directory as the .src folder and not inside it.

### 5.19.1 Attributes

All attributes are initialized once unless otherwise specified. They also should be accessed at class level.

- `String CARDS_FILE_NAME`: A String containing the name of the card's csv file.
- `String CELLS_FILE_NAME`: A String containing the name of the cell's csv file.
- `String MONSTERS_FILE_NAME`: A String containing the name of the monster's csv file.

### 5.19.2 Methods

1. `public static ArrayList<Card> readCards() throws IOException`: Reads the `CARDS_FILE_NAME` CSV file using BufferedReader and creates a type of Card with the values, then adds it into the arraylist that will be returned.
2. `public static ArrayList<Cell> readCells() throws IOException`: Reads the `CELLS_FILE_NAME` CSV file using BufferedReader and creates a type of Cell with the values, then adds it into the arraylist that will be returned.
3. `public static ArrayList<Monster> readMonsters() throws IOException`: Reads the `MONSTERS_FILE_NAME` CSV file using BufferedReader and creates a type of Monster with the values, then adds it into the arraylist that will be returned.

## 5.20 Description of CSV files format

1. Each line represents the information of a single card.
2. The data has no header, i.e. the first line represents the first card.
3. The parameters are separated by a comma (,).

### 5.20.1 Cards

1. The cards are found in a file titled `cards.csv`.
2. The line represents the cards' data as follows: (`cardType, name, description, rarity`) for SwapperCard or ShieldCard OR (`cardType, name, description, rarity, energy`) for EnergyStealCard OR (`cardType, name, description, rarity, lucky`) for StartOverCard OR (`cardType, name, description, rarity, duration`) for ConfusionCard.

### 5.20.2 Cells

1. The cells are found in a file titled `cells.csv`.
2. The line represents the cells' data as follows: (`name, role, energy`) (for DoorCell) OR (`name, effect`) (for TransportCells such as ConveyorBelt or ContaminationSock). The effect can be positive (Conveyor Belt) or negative (Contamination Sock).

### 5.20.3 Monsters

1. The monsters are found in a file titled `monsters.csv`.
2. The line represents the monsters' data as follows: (`monsterType, name, description, role, energy`).

## 5.21 Build the Board Class

**Name :** `Board`

**Package :** `game.engine`

**Type :** Class.

**Description :** A class representing the game board responsible for its cells and cards pile.

### 5.21.1 Attributes

All attributes are **READ ONLY** unless otherwise specified.

1. `Cell[][] boardCells`: A 2D array of cells representing the game board with dimensions `BOARD_ROWS` x `BOARD_COLS`.
2. `ArrayList<Monster> stationedMonsters`: ArrayList containing monsters stationed on the board at monster cells. This attribute is **READ AND WRITE**.
3. `ArrayList<Card> originalCards`: ArrayList containing the original cards read from CSV.
4. `ArrayList<Card> cards`: ArrayList containing the current available cards. This attribute is **READ AND WRITE**.

   All ArrayLists in this class belong to the class itself rather than instances of that class. Their getters and setters should also belong to a class rather than instances of the class

### 5.21.2 Constructors

1. `Board(ArrayList<Card> readCards)`: Constructor that initializes the board cells with dimensions `BOARD_ROWS` x `BOARD_COLS`, initializes the `stationedMonsters` & `cards` as an empty ArrayLists, and sets the `originalCards`.

## 5.22 Build the Game Class

**Name :** `Game`

**Package :** `game.engine`

**Type :** Class.

**Description :** Main game engine class that manages the flow of the game.

### 5.22.1 Attributes

All attributes are **READ ONLY** unless stated otherwise.

1. `Board board`: The game board.
2. `ArrayList<Monster> allMonsters`: List of all available monsters read from the CSV
3. `Monster player`: The player's monster.
4. `Monster opponent`: The opponent's monster.
5. `Monster current`: The monster whose currently playing the turn. This attribute is **READ AND WRITE**.

### 5.22.2 Constructors

1. `Game(Role playerRole) throws IOException`: Constructor that initializes the game by:
   (a) creating the board with the loaded cards from the csv
   (b) setting all monsters with the loaded monsters from the csv
   (c) randomly selecting player based on the player's chosen role
   (d) randomly selecting opponent based on the opposite to player's chosen role
   (e) setting the current with the player

### 5.22.3 Methods

1. `private Monster selectRandomMonsterByRole(Role role)`: randomly returns a monster based on the given role.

# 6 Build the Exceptions

## 6.1 Build the GameActionException Class

**Name :** `GameActionException`

**Package :** `game.engine.exceptions`

**Type :** Abstract class.

**Description :** Class representing a generic exception that can occur during game play. These exceptions arise from any invalid action that is performed. This class is a subclass of java's Exception class.

No objects of type `GameActionException` can be instantiated.

### 6.1.1 Constructors

1. `GameActionException()`: Initializes an instance of a GameActionException by calling the constructor of the super class.
2. `GameActionException(String message)`: Initializes an instance of a GameActionException by calling the constructor of the super class with a customized message.

### 6.1.2 Subclasses

This class has three subclasses: `InvalidMoveException`, `InvalidTurnException` and `OutOfEnergyException`.

## 6.2 Build the InvalidMoveException Class

**Name :** `InvalidMoveException`

**Package :** `game.engine.exceptions`

**Type :** Class

**Description :** Class representing an exception that can occur whenever the player tries to make an invalid move like landing on opponent. This class is a subclass of GameActionException class.

### 6.2.1 Attributes

1. `static final String MSG`: A string representing the message of the exception that should be initialized to `"Invalid move attempted"`.

### 6.2.2 Constructors

1. `InvalidMoveException()`: Initializes an instance of a InvalidMoveException by calling the constructor of the super class with `MSG`.
2. `InvalidMoveException(String message)`: Initializes an instance of a InvalidMoveException by calling the constructor of the super class with a customized message.

## 6.3 Build the InvalidTurnException Class

**Name :** `InvalidTurnException`

**Package :** `game.engine.exceptions`

**Type :** Class

**Description :** Class representing an exception that can occur whenever the player tries to perform an action during the wrong turn. This class is a subclass of GameActionException class.

### 6.3.1 Attributes

1. `static final String MSG`: A string representing the message of the exception that should be initialized to `"Action done on wrong turn"`.

### 6.3.2 Constructors

1. `InvalidTurnException()`: Initializes an instance of a InvalidTurnException by calling the constructor of the super class with `MSG`.
2. `InvalidTurnException(String message)`: Initializes an instance of a InvalidTurnException by calling the constructor of the super class with a customized message.

## 6.4 Build the OutOfEnergyException Class

**Name :** `OutOfEnergyException`

**Package :** `game.engine.exceptions`

**Type :** Class

**Description :** Class representing an exception that can occur whenever the player tries to use monster's power up without having sufficient energy. This class is a subclass of GameActionException class.

### 6.4.1 Attributes

1. `static final String MSG`: A string representing the message of the exception that should be initialized to `"Not Enough Energy for Power Up"`.

### 6.4.2 Constructors

1. `OutOfEnergyException()`: Initializes an instance of a OutOfEnergyException by calling the constructor of the super class with `MSG`.
2. `OutOfEnergyException(String message)`: Initializes an instance of a OutOfEnergyException by calling the constructor of the super class with a customized message.

## 6.5 Build the InvalidCSVFormat Class

**Name :** `InvalidCSVFormat`

**Package :** `game.engine.exceptions`

**Type :** Class

**Description :** Class representing an exception that can occur whenever an invalid csv is being read. This class is a subclass of java's IOException class.

### 6.5.1 Attributes

1. `static final String MSG`: A string representing the message of the exception that should be initialized to `"Invalid input detected while reading csv file, input = \n"`.
2. `String inputLine`: A variable representing the csv file line. This attribute is **READ AND WRITE**.

### 6.5.2 Constructors

1. `InvalidCSVFormat(String inputLine)`: Initializes an instance of a InvalidCSVFormat by calling the constructor of the super class with a message including `MSG` as well as the `inputLine` and setting the `inputLine` with the given parameter.
2. `InvalidCSVFormat(String message, String inputLine)`: Initializes an instance of a InvalidCSVFormat by calling the constructor of the super class with a customized message and setting the `inputLine` with the given parameter.